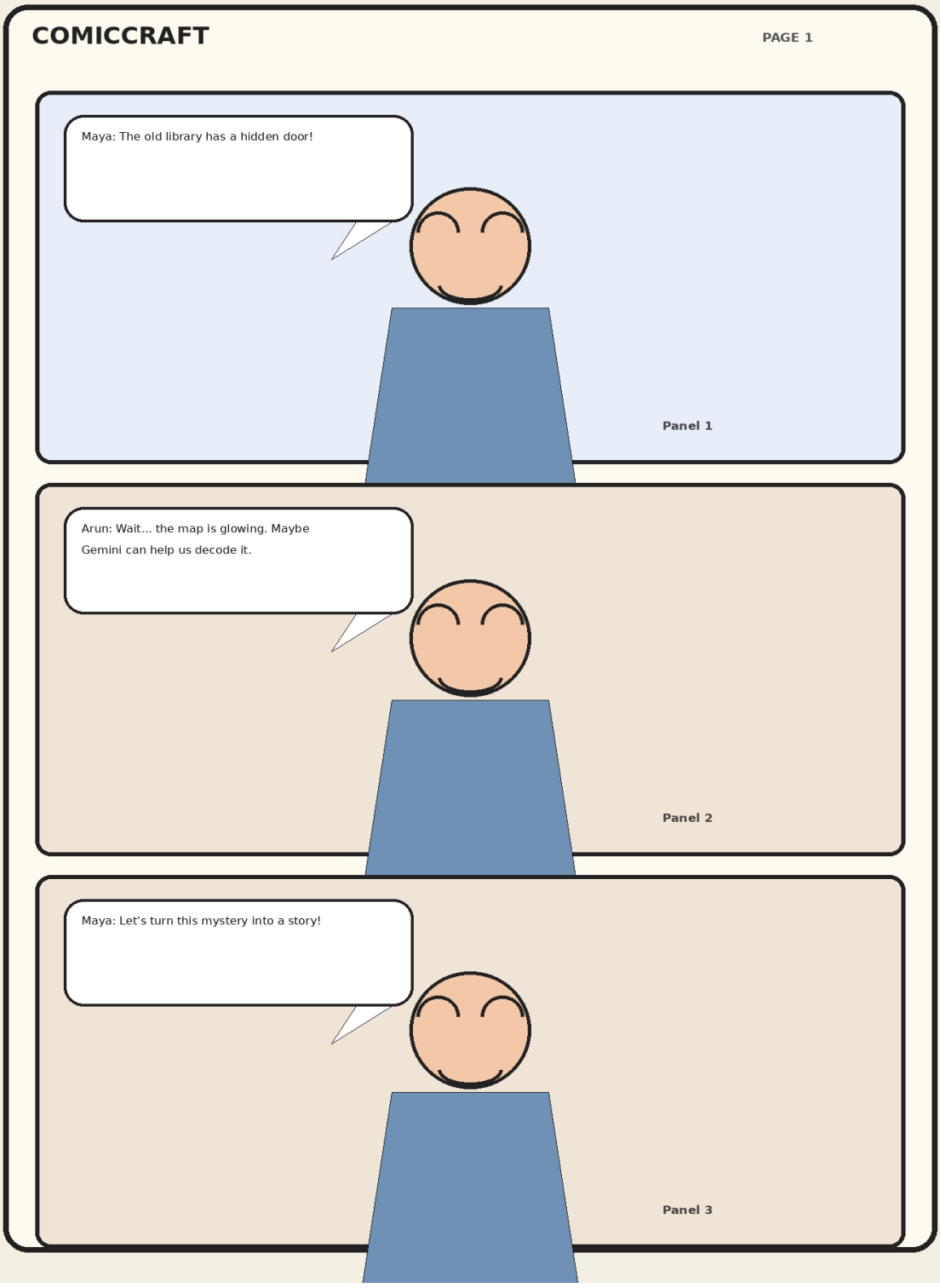
COMICCRAFT

AI Comic Story Creator using Gemini Models

Generative AI Mini Project / Major Project Report

COMICCRAFT

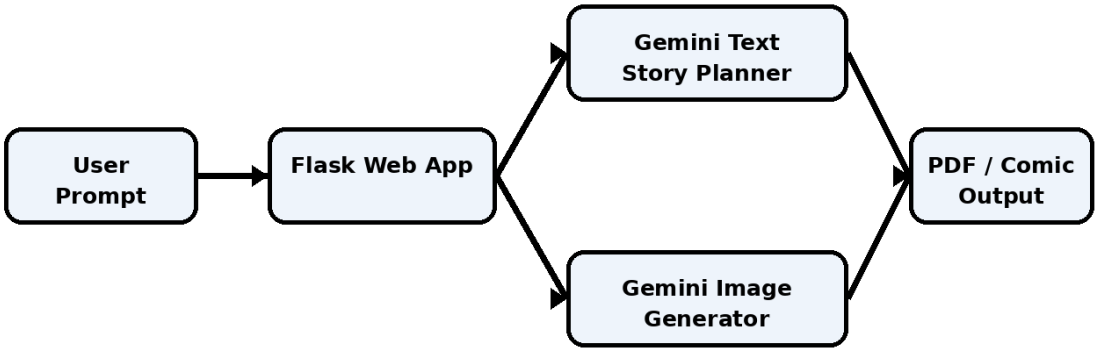
PAGE 1



PROJECT OVERVIEW

Project Title: ComicCraft – AI Comic Story Creator using Gemini Models
Domain: Generative AI / Web Application
Technology: Python, Flask, Gemini API, Pillow, ReportLab, HTML/CSS

This project converts a user's creative idea into a short structured comic story. Gemini is used for story planning and a Gemini image-generation model is used to create comic-style visual panels. The application then assembles the panels and text into a PDF comic.



ABSTRACT

ComicCraft is a Generative AI based web application designed to simplify comic creation. The user provides a topic, genre and page count. The system asks a Gemini text model to create characters, narration, dialogue and panel-level image prompts. These prompts are sent to a Gemini image model to create visual panels. Finally, ReportLab combines the generated assets into a downloadable comic PDF.

The project demonstrates a practical multimodal AI pipeline: **idea** → **story structure** → **image prompts** → **generated images** → **comic layout** → **PDF**. It can be used as an educational demonstration of prompt engineering, API integration, web development and document generation.

KEYWORDS

Generative AI, Gemini API, multimodal AI, prompt engineering, comic generation, Flask, image generation, PDF generation.

1. INTRODUCTION

Traditional comic production requires writing, character design, drawing, panel composition and publishing. These steps can be time-consuming for beginners. Generative AI can assist with these stages by turning natural-language instructions into structured stories and visual concepts.

ComicCraft focuses on a student-friendly workflow. Instead of requiring advanced drawing skills, it provides a simple web form. The application keeps the generated story structured so that each panel can be rendered consistently and placed into a final PDF.

Google's current Gemini API documentation describes Gemini models for text generation and multimodal tasks, while Gemini image-generation models can produce and edit images. The project uses these capabilities as separate text and image stages. ■cite■turn0search6■turn0search1■

Google's current Gemini API documentation describes Gemini models for text generation and multimodal tasks, while Gemini image-generation models can produce and edit images. The project uses these capabilities as separate text and image stages.

2. PROBLEM STATEMENT

Many students have story ideas but do not have the time or artistic skills required to produce a complete comic. A system is needed that can accept a simple idea, generate an original story structure, create visual panels and package the result into a shareable document.

ComicCraft addresses this problem by combining language generation, image generation and PDF composition in one workflow.

3. OBJECTIVES

• Create short original comic stories from natural-language prompts.

• Generate characters, scenes, narration and dialogue automatically.

• Convert panel descriptions into comic-style images using a Gemini image model.

• Provide a simple browser-based interface using Flask.

• Export the generated comic as a PDF for academic/demo use.

• Keep the system modular so text generation and image generation can be changed independently.

4. SYSTEM REQUIREMENTS

Hardware: Dual-core CPU or better, 4 GB RAM minimum, internet connection, standard display and keyboard.
Software: Python 3.10+, Flask, google-genai SDK, Pillow, ReportLab, python-dotenv, modern web browser.
API: Gemini API key stored in an environment variable.

The image-generation documentation currently lists Gemini 2.5 Flash Image and newer Nano Banana image models; the sample code in this project uses the current Gemini 3.1 Flash Image model name as a configurable environment variable, so it can be changed without rewriting the application.
■cite■turn0search0■turn0search1■

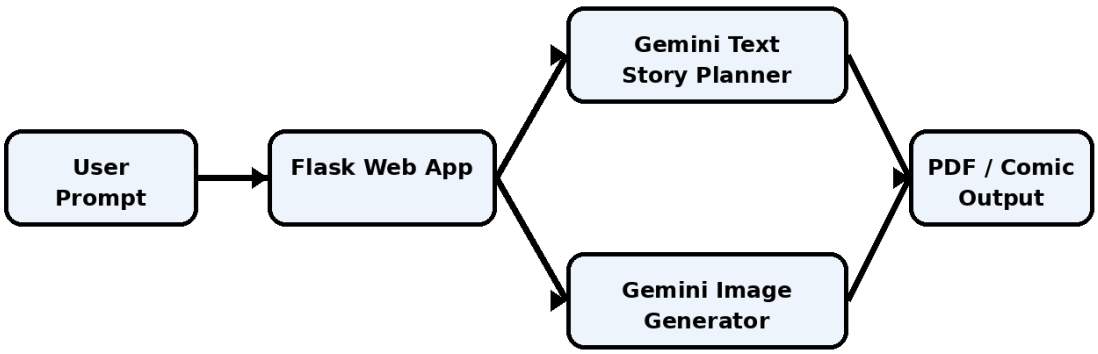
The image-generation documentation currently lists Gemini 2.5 Flash Image and newer Nano Banana image models; the sample code in this project uses the current Gemini 3.1 Flash Image model name as a configurable environment variable, so it can be changed without rewriting the application.

5. MODULES

Module	Purpose
User Interface	Collects topic, genre and page count.
Story Generator	Uses Gemini text generation to create structured JSON.
Prompt Builder	Turns panel requirements into image prompts.
Image Generator	Requests comic-style panel images from Gemini.
Comic Composer	Places images, narration and dialogue into pages.
PDF Exporter	Creates the final downloadable PDF.
File Storage	Stores generated images and story JSON for the current run.

6. SYSTEM WORKFLOW

Step 1 – User enters a topic, genre and page count.
Step 2 – Flask receives the request.
Step 3 – Gemini text model generates title, characters and panel data in JSON format.
Step 4 – Each panel's image prompt is enhanced with comic-style constraints.
Step 5 – Gemini image model generates the panel image.
Step 6 – ReportLab assembles the generated assets into a PDF.
Step 7 – The browser displays the result and provides a PDF download.



7. PROJECT FOLDER STRUCTURE

```
ComicCraft/
■■■ app.py
■■■ requirements.txt
■■■ .env.example
■■■ README.md
■■■ templates/
■ ■■■ index.html
■ ■■■ result.html
■■■ static/
■ ■■■ (CSS / sample assets)
■■■ generated/
■■■ (created automatically at runtime)
```

The generated folder is created automatically. Each story can have its own subfolder containing panel images, story.json and comic_story.pdf.

8. IMPORTANT SOURCE CODE – app.py

```
import os, json, base64, re
from pathlib import Path
from flask import Flask, render_template, request, send_file, flash
from dotenv import load_dotenv
from google import genai
from google.genai import types
from reportlab.lib.pagesizes import A4
from reportlab.pdfgen import canvas
from PIL import Image, ImageDraw, ImageFont

load_dotenv()
app = Flask(__name__)
app.secret_key = os.getenv("FLASK_SECRET", "comiccraft-dev-key")
OUT = Path("generated")
OUT.mkdir(exist_ok=True)

TEXT_MODEL = os.getenv("GEMINI_TEXT_MODEL", "gemini-3.8-flash")
IMAGE_MODEL = os.getenv("GEMINI_IMAGE_MODEL", "gemini-3.1-flash-image")

client = genai.Client(api_key=os.getenv("GEMINI_API_KEY"))

def make_story(topic, genre, pages):
prompt = f"""
Create a short original comic story for a student project.
Topic: {topic}
Genre: {genre}
Pages: {pages}

Return ONLY valid JSON:
{{
"title": "...",
"characters": [
{{"name":"...", "description":"..."}}
],
"panels": [
{{
"page": 1,
"panel": 1,
"scene": "...",
"dialogue": "...",
"narration": "...",
"image_prompt": "..."
}}
]
}}
Make 3 panels per page. Keep the story family-friendly and original.
"""
response = client.models.generate_content(
model=TEXT_MODEL,
contents=prompt,
```

```
config=types.GenerateContentConfig(
response_mime_type="application/json",
temperature=0.9,
),
)
return json.loads(response.text)

def generate_panel(prompt, output_path):
response = client.models.generate_content(
model=IMAGE_MODEL,
contents=[prompt],
config=types.GenerateContentConfig(
response_modalities=["IMAGE"],
),
)
for part in response.candidates[0].content.parts:
if getattr(part, "inline_data", None):
data = part.inline_data.data
if isinstance(data, str):
data = base64.b64decode(data)
output_path.write_bytes(data)
return str(output_path)
raise RuntimeError("No image returned by Gemini image model.")

def create_comic_pdf(story, image_paths, pdf_path):
c = canvas.Canvas(str(pdf_path), pagesize=A4)
W,H = A4

# Cover
c.setFont("Helvetica-Bold", 28)
c.drawCentredString(W/2, H-90, story["title"])
c.setFont("Helvetica", 14)
c.drawCentredString(W/2, H-120, "ComicCraft - AI Comic Story Creator")
y=H-180
for img in image_paths[:1]:
c.drawImage(img, 55, 170, width=W-110, height=H-400, preserveAspectRatio=True, anchor='c')
c.showPage()

# Comic pages
pages = {}
for p in story["panels"]:
pages.setdefault(p["page"], []).append(p)

for page_no, panels in sorted(pages.items()):
c.setFont("Helvetica-Bold", 18)
```

9. WEB INTERFACE CODE

```
&lt;!doctype html&gt;
&lt;html&gt;
&lt;head&gt;&lt;meta charset="utf-8"&gt;&lt;title&gt;ComicCraft&lt;/title&gt;
&lt;style&gt;
body{font-family:Arial;background:#f4efe3;margin:0}.wrap{max-width:850px;margin:60px
auto;background:white;padding:40px;border-radius:22px;box-shadow:0 8px 30px #0002}
h1{font-size:42px}.tag{background:#222;color:white;padding:8px 14px;border-radius:20px;display:inline-block}
label{display:block;margin-top:20px;font-weight:bold}input,select{width:100%;padding:13px;margin-top:7px;border:1px solid
#aaa;border-radius:10px}
button{margin-top:25px;padding:14px 24px;border:0;border-radius:12px;background:#222;color:white;font-size:17px}
&lt;/style&gt;&lt;/head&gt;
&lt;body&gt;&lt;div class="wrap"&gt;&lt;span class="tag"&gt;Gemini Powered&lt;/span&gt;
&lt;h1&gt;ComicCraft&lt;/h1&gt;&lt;p&gt;Turn an idea into an original AI comic story and PDF.&lt;/p&gt;
&lt;form method="post"&gt;
&lt;label&gt;Story topic&lt;/label&gt;&lt;input name="topic" placeholder="A student discovers a time-travel notebook"
required&gt;
&lt;label&gt;Genre&lt;/label&gt;&lt;select
name="genre"&gt;&lt;option&gt;Adventure&lt;/option&gt;&lt;option&gt;Fantasy&lt;/option&gt;&lt;option&gt;Science
Fiction&lt;/option&gt;&lt;option&gt;Comedy&lt;/option&gt;&lt;option&gt;Mystery&lt;/option&gt;&lt;/select&gt;
&lt;label&gt;Pages (1–5)&lt;/label&gt;&lt;input name="pages" type="number" min="1" max="5" value="2"&gt;
&lt;button type="submit"&gt;Create Comic&lt;/button&gt;&lt;/form&gt;&lt;/div&gt;&lt;/body&gt;&lt;/html&gt;
```

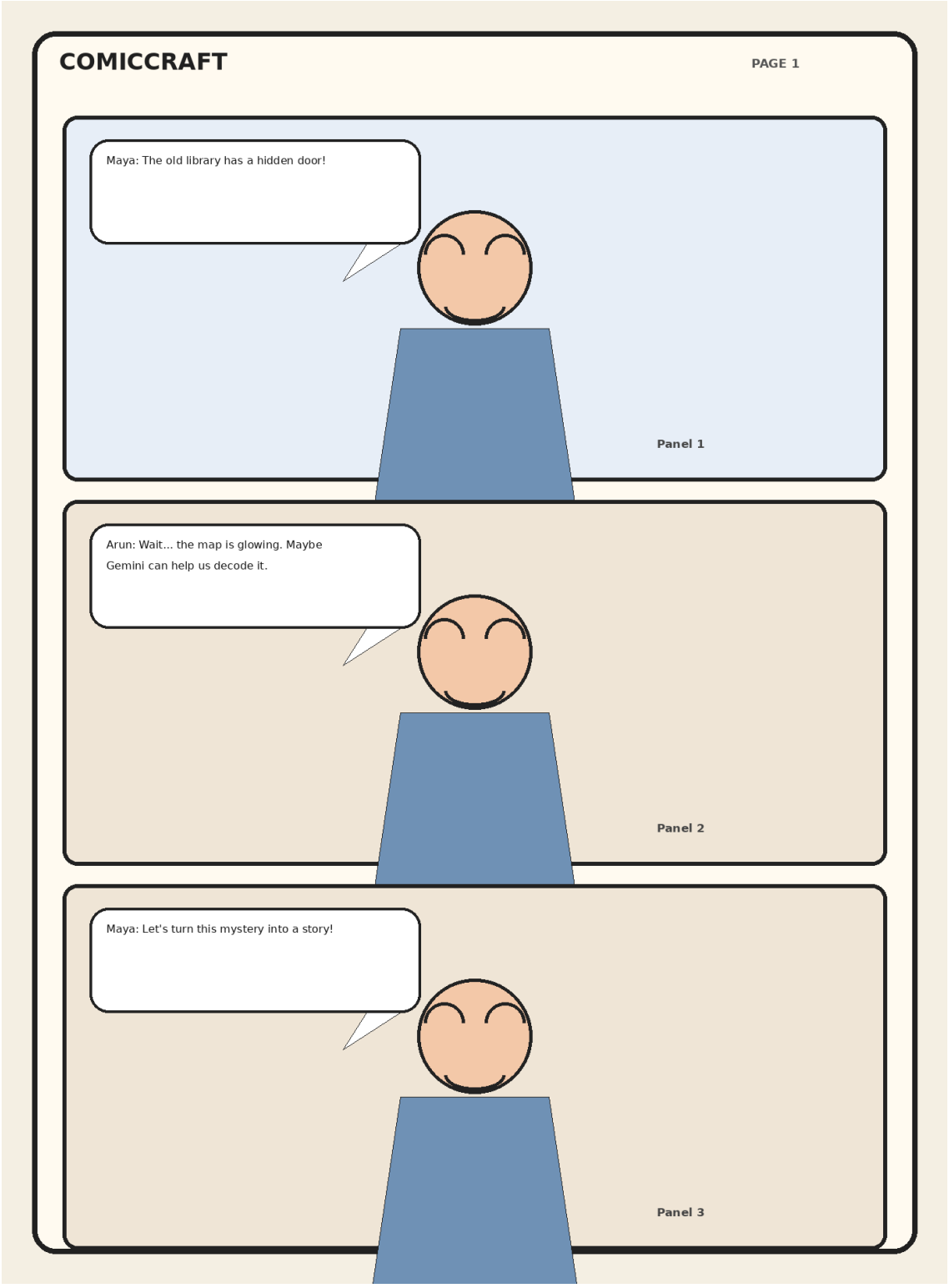
The interface intentionally remains simple so a student can explain the workflow easily during a project viva.

10. CONFIGURATION

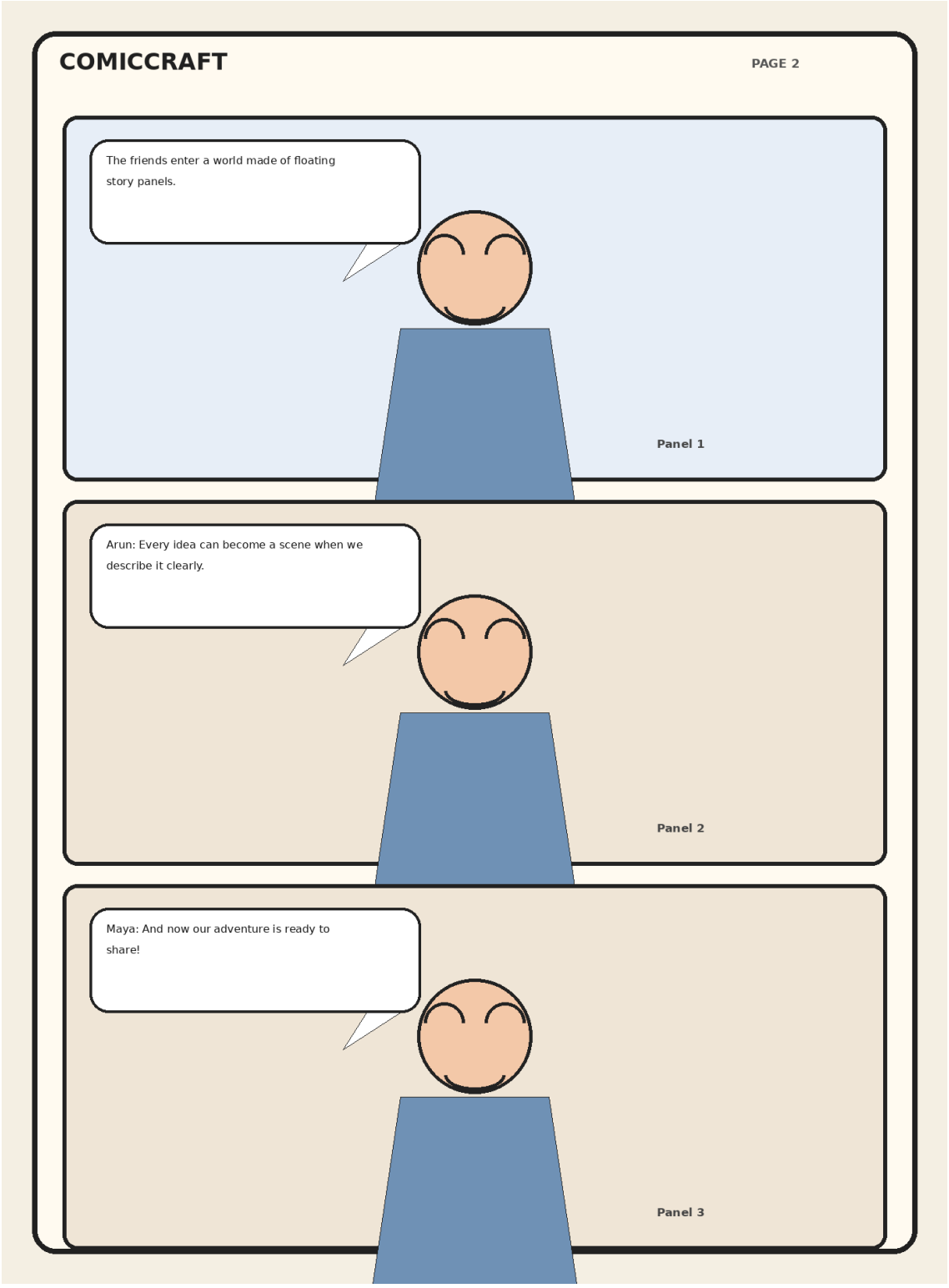
```
GEMINI_API_KEY=your_api_key_here
GEMINI_TEXT_MODEL=gemini-3.8-flash
GEMINI_IMAGE_MODEL=gemini-3.1-flash-image
FLASK_SECRET=change-this-secret
```

Never place a real API key directly in the source code or upload the .env file to a public repository.

11. SAMPLE COMIC OUTPUT



12. SAMPLE COMIC OUTPUT – PAGE 2



13. TESTING

Test Case	Input / Action	Expected Result
TC01	Valid topic + genre + 2 pages	Story JSON is generated.
TC02	Image generation enabled	Panel images are returned and saved.
TC03	PDF export	comic_story.pdf is created.
TC04	Missing API key	Application reports an API/configuration error.
TC05	Page count above 5	Application limits it to 5.
TC06	New topic	A new story folder is created.

14. ADVANTAGES

• Easy for beginners to create a comic from an idea.

• Combines text and image generation in one workflow.

• Structured JSON makes the output easier to process.

• PDF export makes the result suitable for demonstrations and reports.

• Model names are configurable through environment variables.

15. LIMITATIONS

• Generated visual details can vary between runs.

• API access, model availability and pricing depend on the Gemini service.

• Character consistency may require reference-image or iterative workflows.

• Generated content should be reviewed for accuracy, originality and appropriateness.

• Network access is required for API generation.

16. FUTURE ENHANCEMENTS

• Character reference images for stronger consistency.

• Tamil + English story generation and dialogue.

• Voice narration for each panel.

• Editable speech bubbles in the browser.

• Database support for user projects and comic history.

• Cloud deployment and authentication.

• Export to PPTX and image bundles in addition to PDF.

17. CONCLUSION

ComicCraft demonstrates how Generative AI can be integrated into a complete application rather than used only as a chat interface. By separating story planning, visual generation and document composition, the project remains understandable, modular and extensible. It is suitable for a college demonstration of Gemini API integration, prompt engineering, web development and AI-assisted creative content generation.

18. REFERENCES

1. Google AI for Developers – Gemini API: content generation documentation.
2. Google AI for Developers – Gemini API image generation documentation.
3. Flask documentation – Python web application framework.
4. ReportLab documentation – PDF generation library.
5. Pillow documentation – Python Imaging Library.

Note: API model names and capabilities can change. Check Google's current Gemini documentation before deployment. The current documentation identifies Gemini 3.1 Flash Image as a recommended general image-generation option and describes Gemini 2.5 Flash Image as a fast image-generation model. ■cite■turn0search1■turn0search0■

Note: API model names and capabilities can change. Check Google's current Gemini documentation before deployment. The current documentation identifies Gemini 3.1 Flash Image as a recommended general image-generation option and describes Gemini 2.5 Flash Image as a fast image-generation model.